프로그래밍개론 과제 06 보고서

2412975 이서연

#02_double_ll.c

결과

```
Enter [(1) Insert (2) Update (3) Delete (4) Print (5) Exit]:    1
Enter [(1) insert-head (2) insert-tail (3) insert-middle]:      1
ID, value: 1 10
Enter [(1) Insert (2) Update (3) Delete (4) Print (5) Exit]:    1
Enter [(1) insert-head (2) insert-tail (3) insert-middle]:      2
ID, value: 2 30
Enter [(1) Insert (2) Update (3) Delete (4) Print (5) Exit]:    1
Enter [(1) insert-head (2) insert-tail (3) insert-middle]:      3
ID, value: 2 20
Enter [(1) Insert (2) Update (3) Delete (4) Print (5) Exit]:    2
ID for update: 2
Enter the value: 22
Enter [(1) Insert (2) Update (3) Delete (4) Print (5) Exit]:    4
Enter [(1) print-forward (2) print-reverse]:    1
(ID, value): (1, 10)(2, 22)(3, 30)
Enter [(1) Insert (2) Update (3) Delete (4) Print (5) Exit]:    3
ID for delete: 1
Enter [(1) Insert (2) Update (3) Delete (4) Print (5) Exit]:    1
Enter [(1) insert-head (2) insert-tail (3) insert-middle]:      3
ID, value: 2 222
Enter [(1) Insert (2) Update (3) Delete (4) Print (5) Exit]:    4
Enter [(1) print-forward (2) print-reverse]:    2
(ID, value): (3, 30)(2, 222)(1, 22)
Enter [(1) Insert (2) Update (3) Delete (4) Print (5) Exit]:    5
Exit
```

코드 설명

- main 함수

```c
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <stdlib.h>

void dll_insert_head(), dll_insert_tail_head(), dll_insert();
void dll_update(), dll_delete();
void dll_print(), dll_rev_print();

struct NODE {
    struct NODE* prev;
    int ID;
    int value;
    struct NODE* next;
};

struct NODE* head;
struct NODE* tail;
struct NODE* ptr;

int main() {
    head = NULL;
    int index, flag = 0;

    while (flag == 0) {
        printf("Enter [(1) Insert (2) Update (3) Delete (4) Print (5) Exit]:₩t");
        scanf("%d", &index);

        if (index == 1) {
            printf("Enter [(1) insert-head (2) insert-tail (3) insert-middle]:₩t");
            scanf("%d", &index);
            if (index == 1) { dll_insert_head(); }
            else if (index == 2) { dll_insert_tail_head(); }
            else if (index == 3) { dll_insert(); }
            else { printf("enter again (1~3)"); }
        }
        else if (index == 2) { dll_update(); }
        else if (index == 3) { dll_delete(); }
        else if(index == 4){
            printf("Enter [(1) print-forward (2) print-reverse]:₩t");
            scanf("%d", &index);
            if (index == 1) { dll_print(); }
            else if (index == 2) { dll_rev_print(); }
            else{ printf("enter again (1~2)"); }
        }
        else if (index == 5) { printf("Exit₩n"); flag = 1; }
        else { printf("enter again (1~5)"); }
    }
    return 0;
}
```

사용자가 입력한 인덱스에 따라 사용자정의 함수가 사용자의 의도대로 실행될 수 있도록 if, else if문을 이용하였고 사용자정의함수를 각자의 자리에 위치하도록 하였다.

- dll_insert_head()

```c
51  void dll_insert_head()
52  {
53      int insert_ID, insert_value;
54      printf("ID, value: ");
55      scanf("%d %d", &insert_ID, &insert_value);
56      ptr = (struct NODE*)malloc(sizeof(struct NODE));
57      ptr->ID = insert_ID;
58      ptr->value = insert_value;
59      ptr->prev = NULL;
60      ptr->next = NULL;
61
62      if (head == NULL) { tail = ptr; }
63      else { ptr->next = head; head->prev = ptr; }
64      head = ptr;
65
66      while (ptr->next != NULL) {
67          ptr = ptr->next;
68          ptr->ID = ptr->ID + 1;
69      }
70
```

새로운 노드를 리스트의 맨 앞(head)에 삽입하는 함수이다.

56 번 라인의 malloc 함수로 새로운 구조체를 생성하였고 57~60 번 라인으로 값을 초기화하였다.

- 노드 삽입 //62~64
  리스트가 비어 있으면(head == NULL) 새 노드는 head 와 tail 로 설정한다. //62
  비어 있지 않으면 새노드(ptr)의 next 를 기존 head 와 연결하고 기존 head 의 prev 를 새 노드로 설정한다. //63
  새 노드를 새로운 head 로 설정한다. //64

- ID 업데이트 //66~69

  새 노드 이후의 모든 노드의 ID 를 +1 로 재조정한다.

- dll_insert_tail_head()

```c
73  void dll_insert_tail_head()
74  {
75      int insert_ID, insert_value;
76      printf("ID, value: ");
77      scanf("%d %d", &insert_ID, &insert_value);
78      ptr = (struct NODE*)malloc(sizeof(struct NODE));
79      if (head == NULL) { head = ptr; }
80      else { tail->next = ptr; ptr->prev = tail; }
81
82      ptr->ID = insert_ID;
83      ptr->value = insert_value;
84      ptr->next = NULL;
85      ptr->prev = tail;
86      tail = ptr;
87  }
```

새로운 노드를 리스트의 맨 뒤(tail)에 삽입하는 함수이다.

78 번 라인의 malloc 함수로 새로운 구조체를 생성하였고 82~85 번 라인으로 값을 초기화하였다.

- 노드 삽입
  리스트가 비어 있으면(head == NULL) 새 노드는 head 로 설정한다. //79
  비어 있지 않으면 기존 tail 의 next 를 새 노드(ptr)에 연결하고 새 노드의 prev 를 기존 tail 에 연결한다. //80
  새 노드를 새로운 tail 로 설정한다. //86

- dll_insert()

```
89  void dll_insert()
90  {
91      int insert_ID, insert_value;
92      printf("ID, value: ");
93      scanf("%d %d", &insert_ID, &insert_value);
94      ptr = (struct NODE*)malloc(sizeof(struct NODE));
95      ptr->ID = insert_ID;
96      ptr->value = insert_value;
97      ptr->prev = NULL;
98      ptr->next = NULL;
99
100     struct NODE* tmp;
101     tmp = head;
102
103     while (tmp != NULL) {
104         if (tmp->ID == insert_ID) {
105             if (head == NULL) { tail = ptr; }
106             else { ptr->next = head; head->prev = ptr; }
107             head = ptr;
108
109             while (ptr->next != NULL) {
110                 ptr = ptr->next;
111                 ptr->ID = ptr->ID + 1;
112             }
113             return;
114         }
115         else if (tmp->ID == insert_ID - 1) {
116             ptr->next = tmp->next;
117             ptr->prev = tmp;
118
119             if (tmp->next != NULL) { tmp->next->prev = ptr; }
120             else { tail = ptr; }
121
122             tmp->next = ptr;
123             while (ptr->next != NULL) {
124                 ptr = ptr->next;
125                 ptr->ID = ptr->ID + 1;
126             }
127             return;
128         }
129
130         tmp = tmp->next;
131     }
132 }
```

새로운 노드를 특정 위치에 삽입하는 함수이다.

94 번 라인의 malloc 함수로 새로운 구조체를 생성하였고 95~98 번 라인으로 값을 초기화하였다.

- 삽입위치가 맨 앞인 경우 //104~114

  insert_ID 가 1 일 경우를 고려하여 dll_insert_head()함수와 유사한 코드를 if, else if 문을 이용하여 삽입하였다.

- 노드 삽입위치 찾기

  리스트를 순회하며, 삽입할 ID 보다 1 작은 노드를 찾아 위치를 결정하였다. //115

- 노드 삽입
  새 노드의 next 를 tmp(삽입할 ID 보다 1 작은 노드)의 next 값으로 설정. //116
  새 노드의 prev 를 tmp 로 설정. //117
  기존 노드의 prev 를 ptr 로 설정. //119
  tmp 의 next 를 새 노드와 연결. //122
  새 노드가 삽입 위치의 마지막 노드라면 tail 을 업데이트.

- ID 업데이트 //123~126

  새 노드 이후의 모든 노드의 ID 를 +1 로 재조정한다.

- dll_update()

```
134 void dll_update()
135 {
136     int update_ID, update_value;
137     printf("ID for update: ");
138     scanf("%d", &update_ID);
139     ptr = head;
140
141     while (ptr != NULL) {
142         if (ptr->ID == update_ID) {
143             printf("Enter the value: ");
144             scanf("%d", &update_value);
145             ptr->value = update_value;
146             return;
147         }
148         ptr = ptr->next;
149     }
150 }
151
```

특정 ID를 가진 노드의 값을 업데이트하는 함수이다.

리스트를 순회하며 입력받은 update_ID와 일치하는 노드를 찾는다. //142~149

해당 노드를 찾으면 value를 새로 입력받은 값으로 변경한다. //145

- dll_delete()

```
152 void dll_delete()
153 {
154     int delete_ID;
155     printf("ID for delete: ");
156     scanf("%d", &delete_ID);
157     ptr = head;
158
159     if (ptr->ID == delete_ID) {
160         head = ptr->next;
161         head->prev = NULL;
162         head->ID -= 1;
163         free(ptr);
164
165         ptr = head;
166         while (ptr->next != NULL) {
167             ptr = ptr->next;
168             ptr->ID = ptr->ID - 1;
169         }
170         return;
171     }
172
173     while (ptr != NULL) {
174         tail = ptr;
175         ptr = ptr->next;
176         if (ptr->ID == delete_ID) {
177             tail->next = ptr->next;
178
179             if (ptr->next != NULL) { ptr->next->prev = ptr->prev; }
180             free(ptr);
181
182             while (tail->next != NULL) {
183                 tail = tail->next;
184                 tail->ID  = tail->ID - 1;
185             }
186             return;
187         }
188     }
189 }
```

특정 ID를 가진 노드를 삭제하는 함수이다.

- 첫 노드 삭제 //159~171
  head를 다음 노드로 변경. //160
  새로운 head의 prev를 NULL로 설정. //161
  노드 삭제. //163

- 중간~마지막 노드 삭제 //173~188
  삭제할 노드의 이전 노드와 다음 노드를 연결. //117, 179
  노드 삭제. //180

tail은 자동적으로 마지막 노드로 변경되고 next값도 NULL로 변경됨.

- ID 업데이트 //182~185
  삭제한 노드 이후 노드의 ID를 -1씩 조정한다.

- dll_print()

```
191 void dll_print()
192 {
193     ptr = head;
194     printf("(ID, value): ");
195     while (ptr != NULL) {
196         printf("(%d, %d)", ptr->ID, ptr->value);
197         ptr = ptr->next;
198     }
199     printf("₩n");
200     return;
201 }
```

리스트를 정방향으로 출력하는 함수이다.

head부터 시작하여 리스트를 순회한다. ptr을 ptr->next로 옮겨가면서 순회한다. //197

각 노드의 ID와 value를 출력한다. //196

- dll_rev_print()

```
203 void dll_rev_print()
204 {
205     ptr = tail;
206     printf("(ID, value): ");
207     while (ptr != NULL) {
208         printf("(%d, %d)", ptr->ID, ptr->value);
209         ptr = ptr->prev;
210     }
211     printf("₩n");
212     return;
213 }
```

리스트를 역방향으로 출력하는 함수이다.

tail부터 시작하여 리스트를 순회한다. ptr을 ptr->prev로 옮겨가면서 순회한다. //209

각 노드의 ID와 value를 출력한다. //208